

Docker Cheat Sheet

Container Commands

Command	Purpose
<code>docker run -it ubuntu bash</code>	Run a container interactively with a Bash shell.
<code>docker run -d nginx</code>	Run a container in detached mode.
<code>docker ps</code>	List running containers.
<code>docker ps -a</code>	List all containers, including stopped containers.
<code>docker stop <container></code>	Stop a running container.
<code>docker rm <container></code>	Remove a stopped container.
<code>docker exec -it <container> bash</code>	Open an interactive shell inside a running container.
<code>docker logs <container></code>	View container logs.

Image Commands

Command	Purpose
<code>docker build -t <name>:<tag> .</code>	Build an image from a Dockerfile and assign a name and tag.
<code>docker pull <image></code>	Download an image from a container registry.
<code>docker push <username>/<repo>:<tag></code>	Push a tagged image to a container registry.
<code>docker tag <image> <name>:<tag></code>	Create another tag for an existing image.
<code>docker images</code>	List local Docker images.
<code>docker rmi <image></code>	Remove a local Docker image.

Volume Commands

Command	Purpose
<code>docker volume create <volume></code>	Create a named volume managed by Docker.
<code>docker volume ls</code>	List Docker volumes.
<code>docker volume inspect <volume></code>	Display detailed volume information.
<code>docker volume rm <volume></code>	Remove a Docker volume.

Network Commands

Command	Purpose
<code>docker network create <network></code>	Create a custom Docker network.
<code>docker network ls</code>	List Docker networks.
<code>docker network inspect <network></code>	Display network configuration and connected containers.
<code>docker network connect <network> <container></code>	Connect an existing container to a network.

Compose Commands

Command	Purpose
<code>docker compose up -d</code>	Create and start Compose services in detached mode.
<code>docker compose down</code>	Stop and remove Compose services and networks.
<code>docker compose ps</code>	Display the status of Compose services.
<code>docker compose logs -f</code>	Follow logs from Compose services.

Command	Purpose
<code>docker compose build</code>	Build images defined in the Compose file.

Cleanup Commands

Command	Purpose
<code>docker system df</code>	Display Docker disk usage.
<code>docker system prune</code>	Remove unused Docker resources.
<code>docker system prune -a</code>	Remove unused resources, including unused images; use with caution.

Dockerfile Instructions

Command	Purpose
<code>FROM <image></code>	Set the base image.
<code>RUN <command></code>	Execute a command during the image build.
<code>COPY <src> <dest></code>	Copy files or directories from the build context into the image.
<code>WORKDIR <path></code>	Set the working directory for subsequent instructions.
<code>EXPOSE <port></code>	Document the port used by the application; it does not publish the port.
<code>CMD ["command", "arg"]</code>	Set the default command or arguments for the container.
<code>ENTRYPOINT ["command"]</code>	Define the main executable of the container.

Revision Questions & Answers

1. How do you run a container from Docker Hub in interactive mode?

Answer: `docker run -it ubuntu bash` starts an Ubuntu container interactively and opens a Bash shell.

2. How do you run a container from Docker Hub in detached mode?

Answer: `docker run -d nginx` starts an Nginx container in the background.

3. How do you list, stop, and remove containers?

Answer: Use `docker ps` or `docker ps -a` to list containers, `docker stop <container>` to stop one, and `docker rm <container>` to remove a stopped container.

4. How do you list and remove Docker images?

Answer: Use `docker images` to list local images and `docker rmi <image>` to remove an image.

5. What are Docker image layers?

Answer: Docker images are built from multiple read-only layers. Filesystem changes from Dockerfile instructions are represented in layers that can be reused by builds.

6. How does Docker build caching work?

Answer: Docker reuses previously built layers when the relevant Dockerfile instruction and inputs have not changed. Once a layer's cache is invalidated, dependent layers generally need to be rebuilt.

7. Which Dockerfile instructions are commonly used when writing a Dockerfile from scratch?

Answer: FROM selects the base image; RUN executes build-time commands; COPY copies files; WORKDIR sets the working directory; EXPOSE documents a port; and CMD defines the default command.

8. What is the difference between CMD and ENTRYPOINT?

Answer: CMD provides default commands or arguments and can be overridden at runtime. ENTRYPOINT defines the main executable. When combined, CMD commonly supplies default arguments to the ENTRYPOINT.

9. How do you build and tag a custom Docker image?

Answer: Use `docker build -t my-nginx:v1 .` to build an image and assign the my-nginx:v1 tag.

10. How do you create and use a named volume?

Answer: Create it with `docker volume create mydata`, then attach it with `docker run -v mydata:/data <image>`.

11. What is a bind mount and how is it used?

Answer: A bind mount maps a host filesystem path directly into a container, for example `docker run -v $(pwd):/app <image>`.

12. How do you create a custom Docker network?

Answer: Use `docker network create app-network`.

13. How do containers communicate on the same custom network?

Answer: Containers on the same custom network can communicate using container or service names through Docker's built-in DNS, rather than relying on container IP addresses.

14. How do you connect an existing container to a custom network?

Answer: Use `docker network connect <network> <container>`.

15. How do you define and start a multi-container application with Docker Compose?

Answer: Define multiple services under `services:` in `docker-compose.yml`, then run `docker compose up -d`.

16. How are environment variables and `.env` files used in Compose?

Answer: Values can be defined in `.env` and referenced with `${VARIABLE_NAME}`. Use `docker compose config` to inspect the resolved configuration.

17. Why are multi-stage Dockerfiles useful?

Answer: They separate the build environment from the runtime environment so the final image can contain only the required application artifacts and runtime dependencies.

18. How do you push an image to Docker Hub?

Answer: Tag the image with the Docker Hub repository name using `docker tag`, then push it using `docker push <username>/<repository>:<tag>`.

19. What is a Docker healthcheck?

Answer: A healthcheck defines a command or test that Docker uses to determine whether an application inside a container is healthy.

20. What is `depends_on` used for in Docker Compose?

Answer: `depends_on` defines service dependencies. With health-based dependency conditions, a dependent service can wait for another service to become healthy.

21. What is the difference between a Docker image and a container?

Answer: An image is a read-only template containing application code, dependencies, and configuration. A container is an instance created from an image.

22. What happens to data when a container is removed?

Answer: Data stored only in the container's writable layer is removed with the container. Data stored in a named volume or bind mount persists separately.

23. How do `docker compose down` and `docker compose down -v` differ?

Answer: `docker compose down` removes Compose containers and networks. `docker compose down -v` also removes the volumes associated with the Compose project.

24. What is the difference between COPY and ADD?

Answer: COPY performs straightforward file and directory copying. ADD provides additional functionality, such as handling local tar archives. COPY is generally preferred for normal file copying.

25. What does `-p 8080:80` mean?

Answer: It maps host port 8080 to container port 80. The application inside the container must actually listen on port 80.

26. How do you check Docker disk space usage?

Answer: Use `docker system df` to display Docker disk usage for images, containers, volumes, and build cache.

Practical Reference

Named Volume

```
docker volume create mydata
docker run -v mydata:/data <image>
```

Docker-managed persistent storage that remains independent of the container.

Bind Mount

```
docker run -v $(pwd):/app <image>
```

Maps a host directory directly into the container.

Port Mapping

```
docker run -p 8080:80 nginx
```

Maps host port 8080 to container port 80.

Compose Service Discovery

```
http://backend:8080
```

Services on the same Compose network can normally reach each other by service name.

Healthcheck

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

Defines how Docker determines whether an application inside the container is healthy.

Port Mapping Reminder

Important: `EXPOSE 80` documents the container port but does not publish it. The application must actually listen on the mapped container port. For an application listening on container port `4173`, a mapping such as `8080:4173` is required to expose it through host port `8080`.

Quick Revision Commands

```
# Containers
docker run -it ubuntu bash
docker run -d nginx
docker ps
docker ps -a
docker stop <container>
docker rm <container>
docker exec -it <container> bash
docker logs <container>

# Images
docker images
docker pull <image>
docker build -t <name>:<tag> .
docker tag <image> <name>:<tag>
docker push <username>/<repo>:<tag>
docker rmi <image>

# Volumes
docker volume create <volume>
docker volume ls
docker volume inspect <volume>
docker volume rm <volume>

# Networks
docker network create <network>
docker network ls
docker network inspect <network>
docker network connect <network> <container>

# Compose
docker compose up -d
docker compose down
docker compose ps
docker compose logs -f
docker compose build
```

```
# Cleanup
docker system df
docker system prune
docker system prune -a
```